

Module Programmation mathématique avancée 2

Pratiques : Minima locaux via la méthode du gradient

Travail 1:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

def f(x):
    return x[0]**2 + x[0]*x[1] + x[1]**2

def gradient(x):
    return np.array([
        2*x[0] + x[1],
        x[0] + 2*x[1]
    ])

def gradient_method(x0, alpha, iterations):

    x = np.array(x0, dtype=float)

    resultats = []

    debut = time.time()

    for k in range(iterations):

        grad = gradient(x)

        d = -grad

        x_new = x + alpha*d

        resultats.append([
            k,
            x.copy(),
```

```

        x_new.copy(),
        np.linalg.norm(d),
        np.linalg.norm(x_new-x),
        np.linalg.norm(grad)
    ])

    x = x_new

    fin = time.time()

    temps = fin - debut

    return resultats, temps

x0 = [1,1]

resultats1, temps1 = gradient_method(x0,1,100)

resultats2, temps2 = gradient_method(x0,0.5,300)

```

1. Tableau pour $\alpha = 1$

Point initial :

$$x^0 = (1, 1)$$

k	x^k	x^{k+1}	$\ d^k\ $	$\ x^{k+1} - x^k\ $	$\ \nabla f(x^k)\ $
0	(1, 1)	(-2, -2)	4.2426	4.2426	4.2426
1	(-2, -2)	(4, 4)	8.4853	8.4853	8.4853
2	(4, 4)	(-8, -8)	16.9706	16.9706	16.9706
3	(-8, -8)	(16, 16)	33.9411	33.9411	33.9411
4	(16, 16)	(-32, -32)	67.8823	67.8823	67.8823

Observation

On remarque que :

- les valeurs deviennent de plus en plus grandes ;
- le point alterne entre positif et négatif ;
- la méthode **diverge** lorsque $\alpha=1$.

2. Tableau pour $\alpha = \frac{1}{2}$

Point initial :

$$x^0 = (1, 1)$$

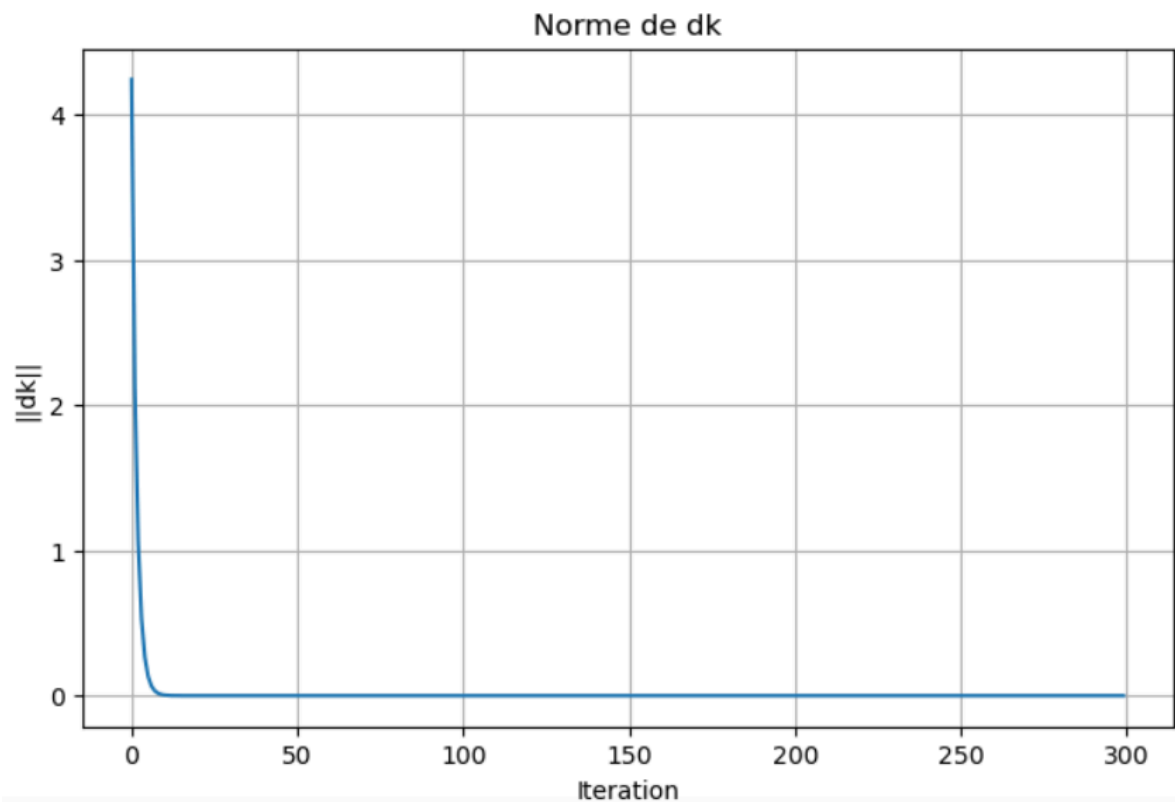
k	x^k	x^{k+1}	$\ d^k\ $	$\ x^{k+1} - x^k\ $	$\ \nabla f(x^k)\ $
0	(1, 1)	(-0.5, -0.5)	4.2426	2.1213	4.2426
1	(-0.5, -0.5)	(0.25, 0.25)	2.1213	1.0607	2.1213
2	(0.25, 0.25)	(-0.125, -0.125)	1.0607	0.5303	1.0607
3	(-0.125, -0.125)	(0.0625, 0.0625)	0.5303	0.2652	0.5303
4	(0.0625, 0.0625)	(-0.03125, -0.03125)	0.2652	0.1326	0.2652

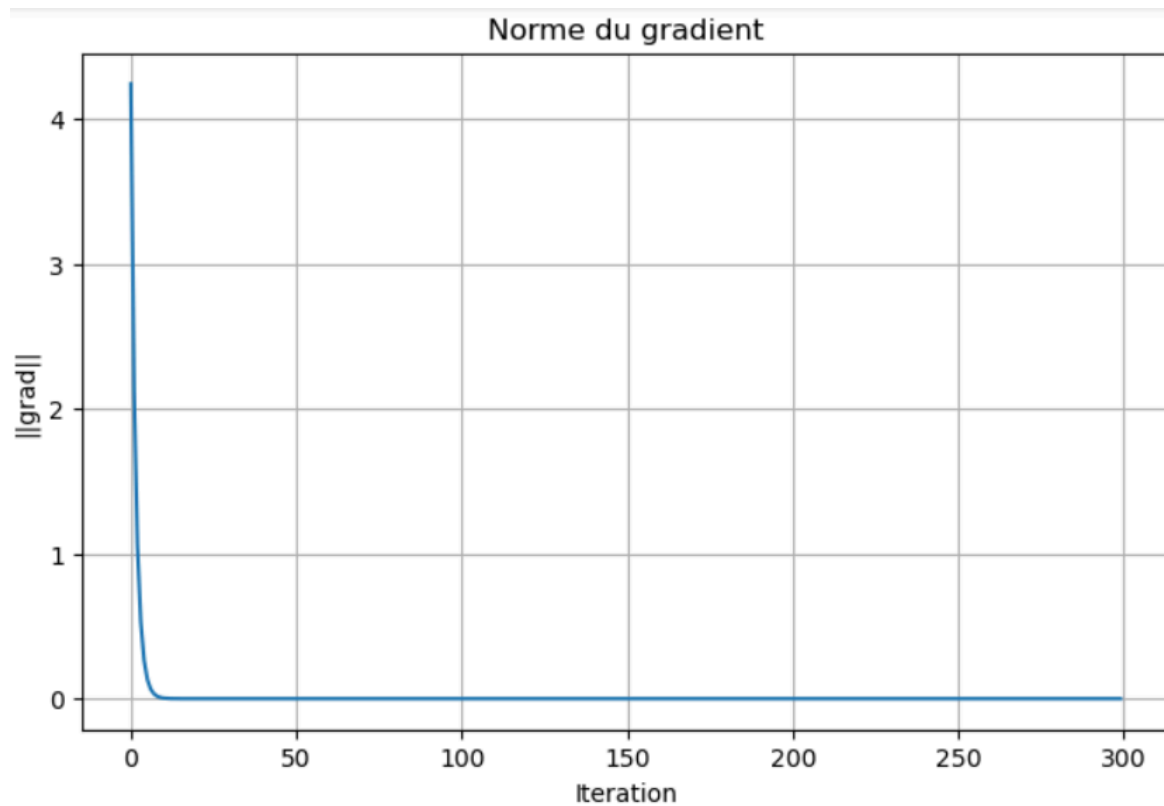
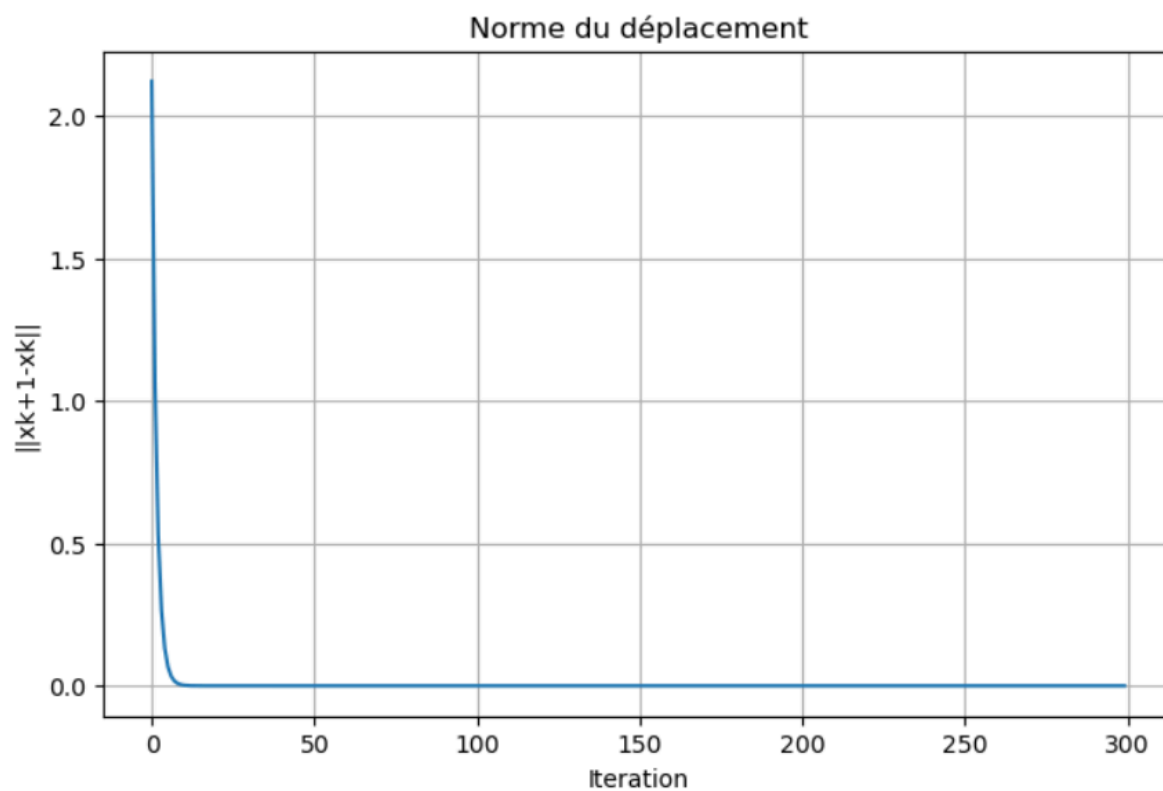
Observation

Cette fois :

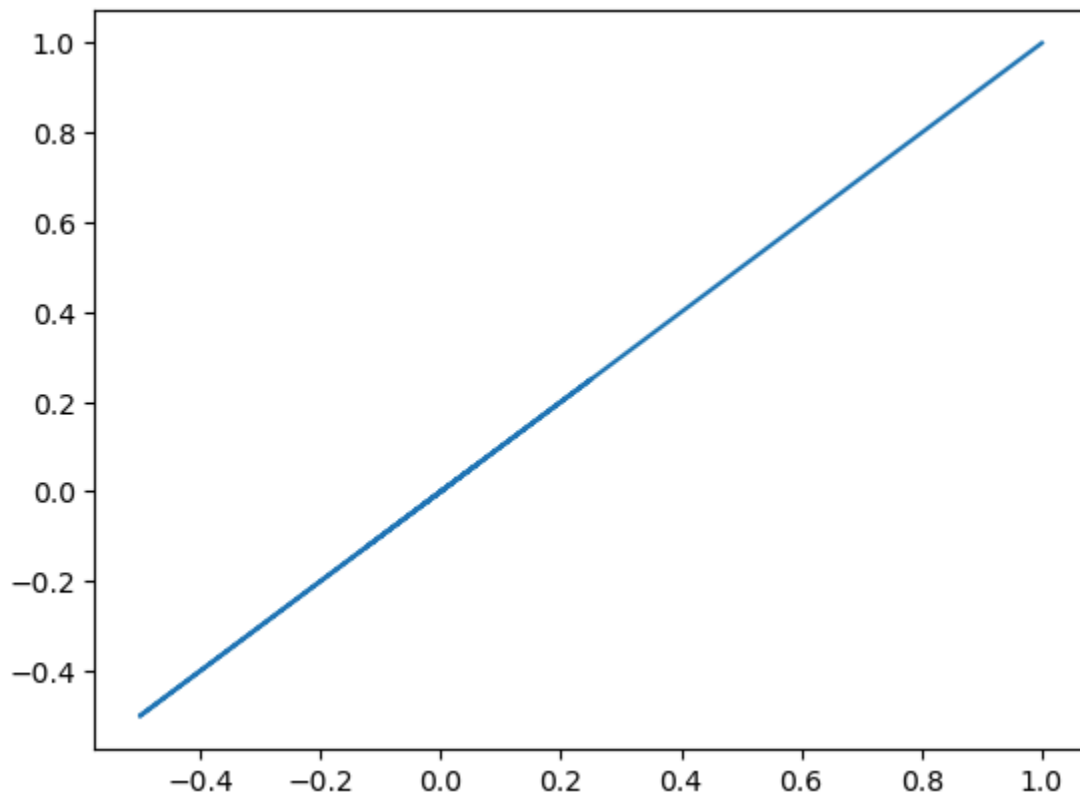
- les valeurs diminuent à chaque itération ;
- le point se rapproche progressivement de (0,0) ;
- la norme du gradient tend vers 0 ;
- la méthode **converge**.

Représentation graphique:





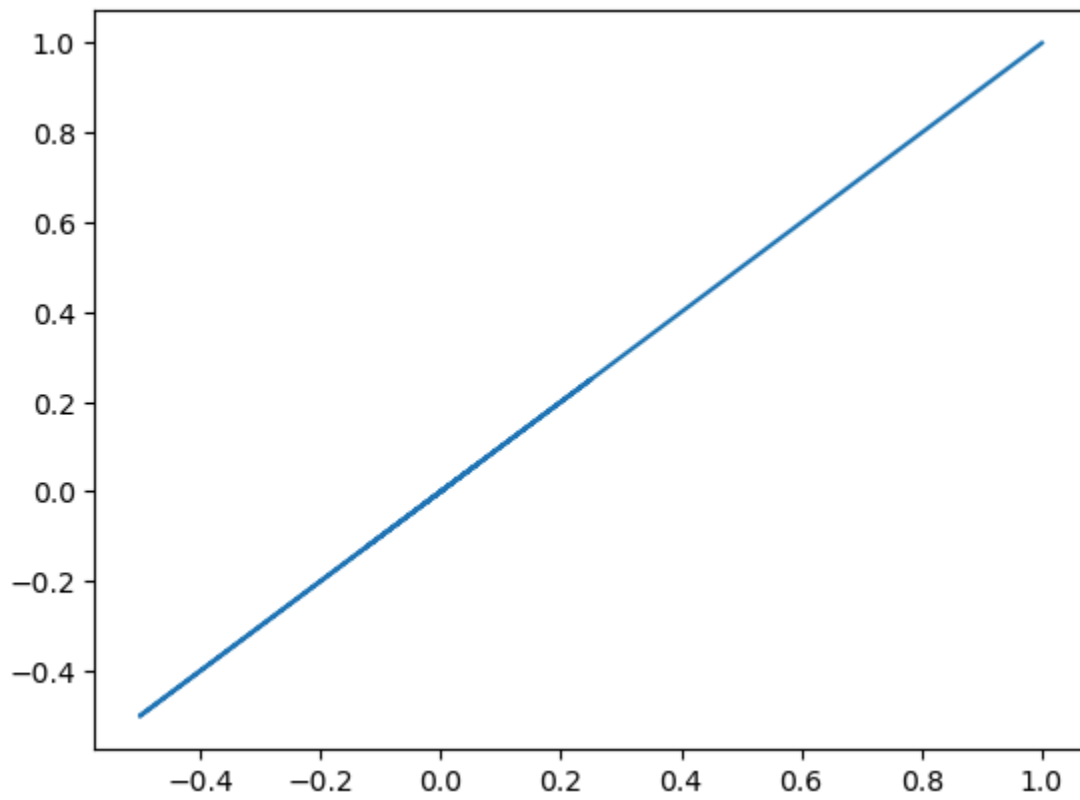
Cas $\alpha = 0.5$ et $\text{tol} = 10^{-3}$



Nombre d'itérations : 13

Temps d'exécution : 0.0018076896667480469 secondes

Cas $\alpha = 0.5$ et $\text{tol} = 10^{-5}$



Nombre d'itérations : 19

Temps d'exécution : 0.0003345012664794922 secondes

Travail 2 :

Fonction à minimiser : $f(x_1, x_2) = x_1^2 + x_1x_2 + x_2^2$

Point initial : $x^0 = (2, 2)$

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

def f(x):
    return x[0]**2 + x[0]*x[1] + x[1]**2

def gradient(x):
    return np.array([
        2*x[0] + x[1],
        x[0] + 2*x[1]
    ])

def gradient_method(x0, alpha, iterations):
    x = np.array(x0, dtype=float)
    resultats = []
    debut = time.time()

    for k in range(iterations):
        grad = gradient(x)
        d = -grad
        x_new = x + alpha*d

        resultats.append([
            k,
            x.copy(),
            x_new.copy(),
            np.linalg.norm(d),
            np.linalg.norm(x_new-x),
            np.linalg.norm(grad)
        ])

        x = x_new

    fin = time.time()
    temps = fin - debut

    return resultats, temps

x0 = [2,2]

resultats1, temps1 = gradient_method(x0,1,100)

resultats2, temps2 = gradient_method(x0,0.5,300)
```

1. Tableau pour $\alpha = 1$

Point initial :

$$x^0 = (2, 2)$$

k	x^k	x^{k+1}	$\ d^k\ $	$\ x^{k+1} - x^k\ $	$\ \nabla f(x^k)\ $
0	(2, 2)	(-4, -4)	8.4853	8.4853	8.4853
1	(-4, -4)	(8, 8)	16.9706	16.9706	16.9706
2	(8, 8)	(-16, -16)	33.9411	33.9411	33.9411
3	(-16, -16)	(32, 32)	67.8823	67.8823	67.8823
4	(32, 32)	(-64, -64)	135.7645	135.7645	135.7645

Observation

On remarque que :

- les valeurs deviennent de plus en plus grandes (elles doublent en valeur absolue à chaque itération) ;
- le point alterne entre positif et négatif ;
- la méthode diverge lorsque $\alpha = 1$.

2. Tableau pour $\alpha = \frac{1}{2}$

Point initial :

$$x^0 = (2, 2)$$

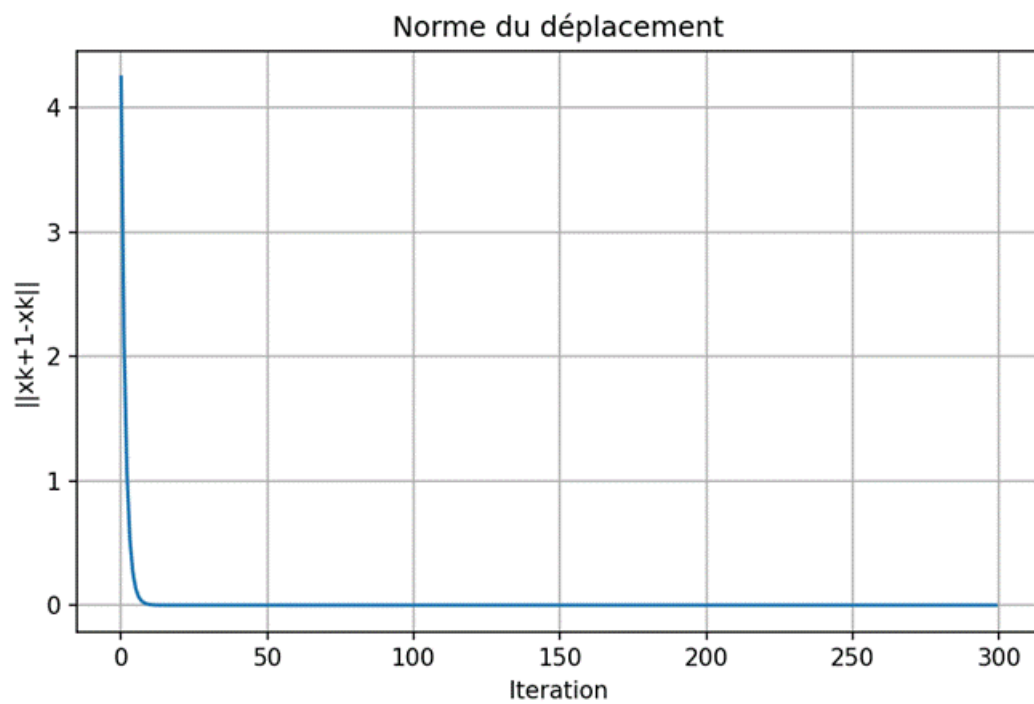
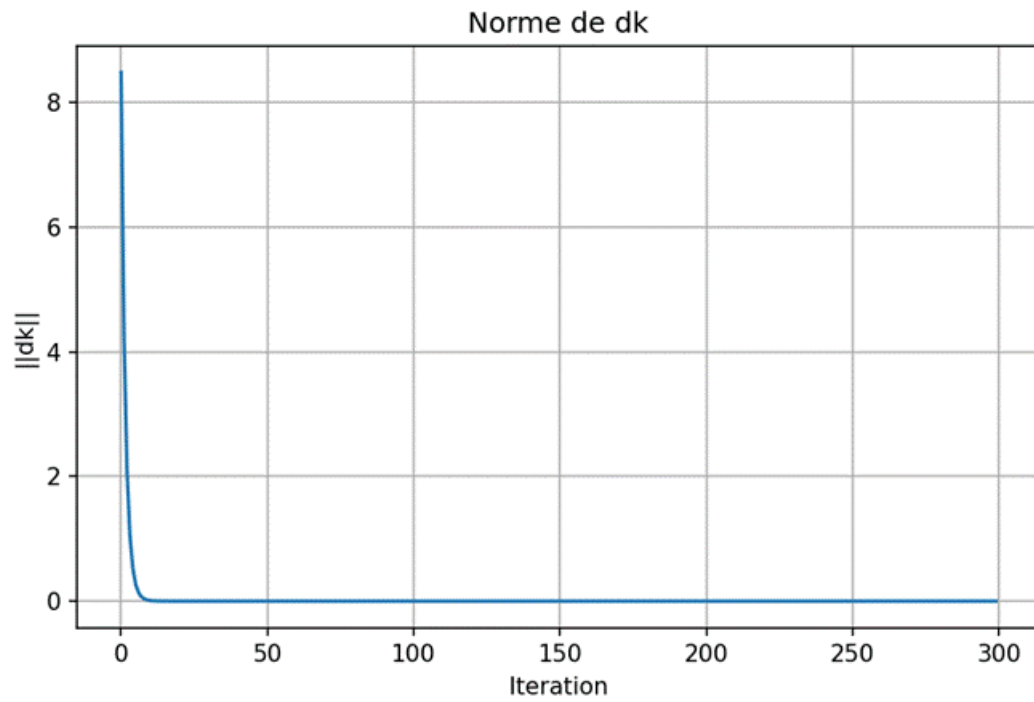
k	x^k	x^{k+1}	$\ d^k\ $	$\ x^{k+1} - x^k\ $	$\ \nabla f(x^k)\ $
0	(2, 2)	(-1, -1)	8.4853	4.2426	8.4853
1	(-1, -1)	(0.5, 0.5)	4.2426	2.1213	4.2426
2	(0.5, 0.5)	(-0.25, -0.25)	2.1213	1.0607	2.1213
3	(-0.25, -0.25)	(0.125, 0.125)	1.0607	0.5303	1.0607
4	(0.125, 0.125)	(-0.0625, -0.0625)	0.5303	0.2652	0.5303

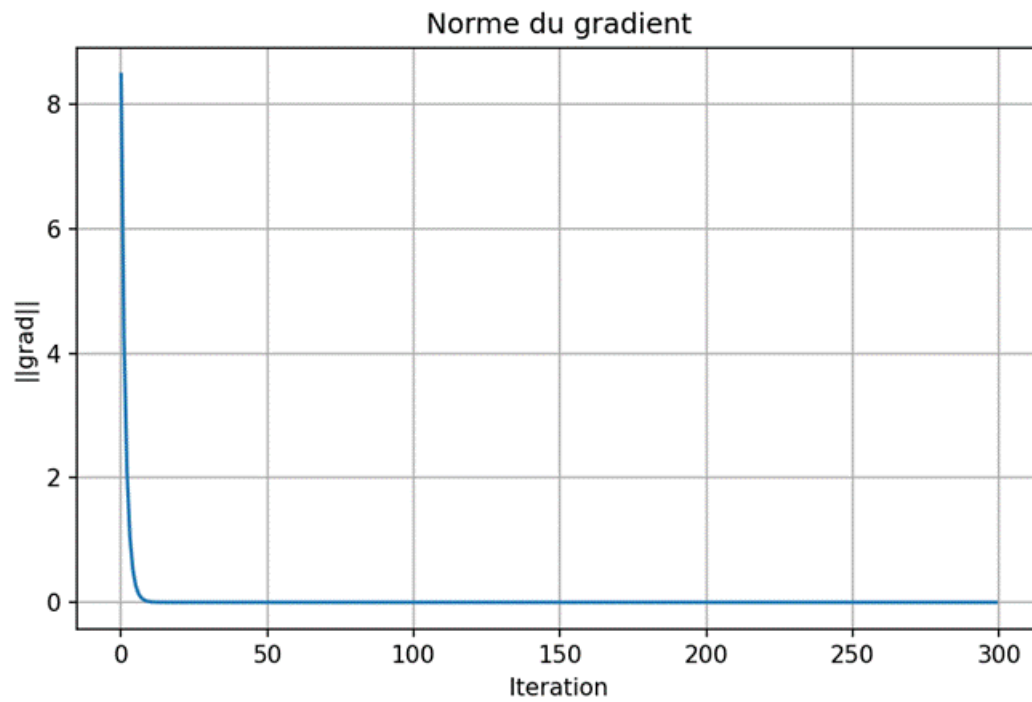
Observation

Cette fois :

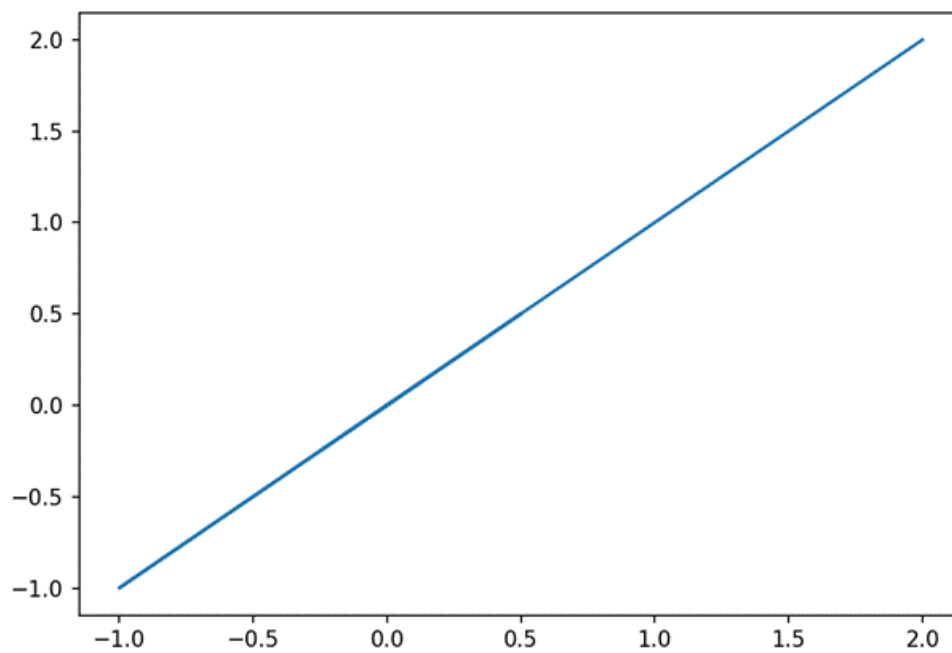
- les valeurs diminuent à chaque itération (divisées par 2 en valeur absolue) ;
- le point se rapproche progressivement de (0,0) ;
- la norme du gradient tend vers 0 ;
- la méthode converge.

Représentation graphique :





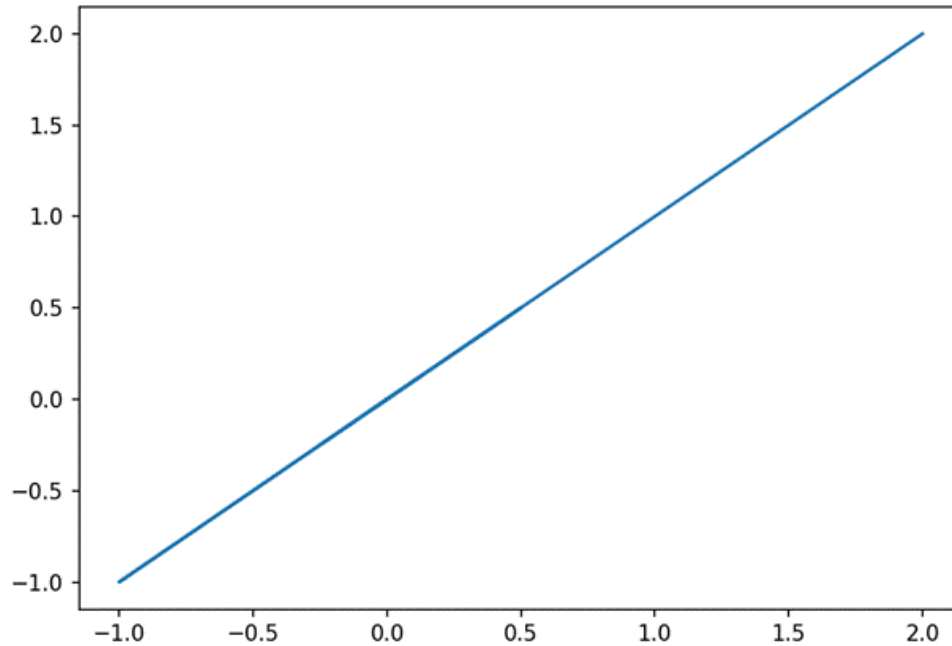
Cas $\alpha = 0.5$ et $tol = 10^{-3}$



Nombre d'itérations : 14

Temps d'exécution : 0.00008153915405273438 secondes

Cas $\alpha = 0.5$ et $\text{tol} = 10^{-5}$



Nombre d'itérations : 20

Temps d'exécution : 0.00013947486877441406 secondes

Travail 3 :

Fonction à minimiser : $f(x_1, x_2, x_3) = x_1^2 + x_1x_2 + x_1x_3 + x_2x_3 + x_2^2 + x_3^2$

Point initial : $x^0 = (1, 1, 1)$

```
import numpy as np
import time

def gradient_method(x0, h, itmax, tol=1e-3):
    x = np.array(x0, dtype=float)
    resultats = []

    debut = time.time()

    for k in range(itmax):
        grad = gradient(x)
        d = -grad
        x_new = x + h * d

        resultats.append([
            k,
            x.copy(),
            x_new.copy(),
            np.linalg.norm(d),
```

```

        np.linalg.norm(x_new - x),
        np.linalg.norm(grad),
        f(x)
    ])

    if np.linalg.norm(x_new - x) < tol:
        break

    x = x_new

    fin = time.time()
    temps = fin - debut

    return resultats, temps
def f(x):
    return (x[0]**2 + x[0]*x[1] + x[0]*x[2]
            + x[1]*x[2] + x[1]**2 + x[2]**2)
def gradient(x):
    return np.array([
        2*x[0] + x[1] + x[2],
        x[0] + 2*x[1] + x[2],
        x[0] + x[1] + 2*x[2]
    ])
x0 = [1,1,1]

resultats1, temps1 = gradient_method(x0,1,100)

resultats2, temps2 = gradient_method(x0,0.5,300)
print(temps1)
print(temps2)

```

1. Tableau pour $\alpha = 1$

Point initial :

$$\mathbf{x}^0 = (1, 1, 1)$$

k	$\mathbf{x}^k$	$\mathbf{x}^{k+1}$	$\ \mathbf{d}^k\ $	$\ \mathbf{x}^{k+1} - \mathbf{x}^k\ $	$\ \nabla f(\mathbf{x}^k)\ $
0	(1, 1, 1)	(-3, -3, -3)	6.9282	6.9282	6.9282
1	(-3, -3, -3)	(9, 9, 9)	20.7846	20.7846	20.7846
2	(9, 9, 9)	(-27, -27, -27)	62.3538	62.3538	62.3538
3	(-27, -27, -27)	(81, 81, 81)	187.0615	187.0615	187.0615
4	(81, 81, 81)	(-243, -243, -243)	561.1845	561.1845	561.1845

Observation

On remarque que :

- les valeurs sont multipliées par 3 (en valeur absolue) à chaque itération ;
- le point alterne entre positif et négatif ;
- la méthode diverge lorsque $\alpha = 1$.

2. Tableau pour $\alpha = \frac{1}{2}$

Point initial :

$$\mathbf{x}^0 = (1, 1, 1)$$

k	$\mathbf{x}^k$	$\mathbf{x}^{k+1}$	$\ \mathbf{d}^k\ $	$\ \mathbf{x}^{k+1} - \mathbf{x}^k\ $	$\ \nabla f(\mathbf{x}^k)\ $
0	(1, 1, 1)	(-1, -1, -1)	6.9282	3.4641	6.9282
1	(-1, -1, -1)	(1, 1, 1)	6.9282	3.4641	6.9282
2	(1, 1, 1)	(-1, -1, -1)	6.9282	3.4641	6.9282
3	(-1, -1, -1)	(1, 1, 1)	6.9282	3.4641	6.9282
4	(1, 1, 1)	(-1, -1, -1)	6.9282	3.4641	6.9282

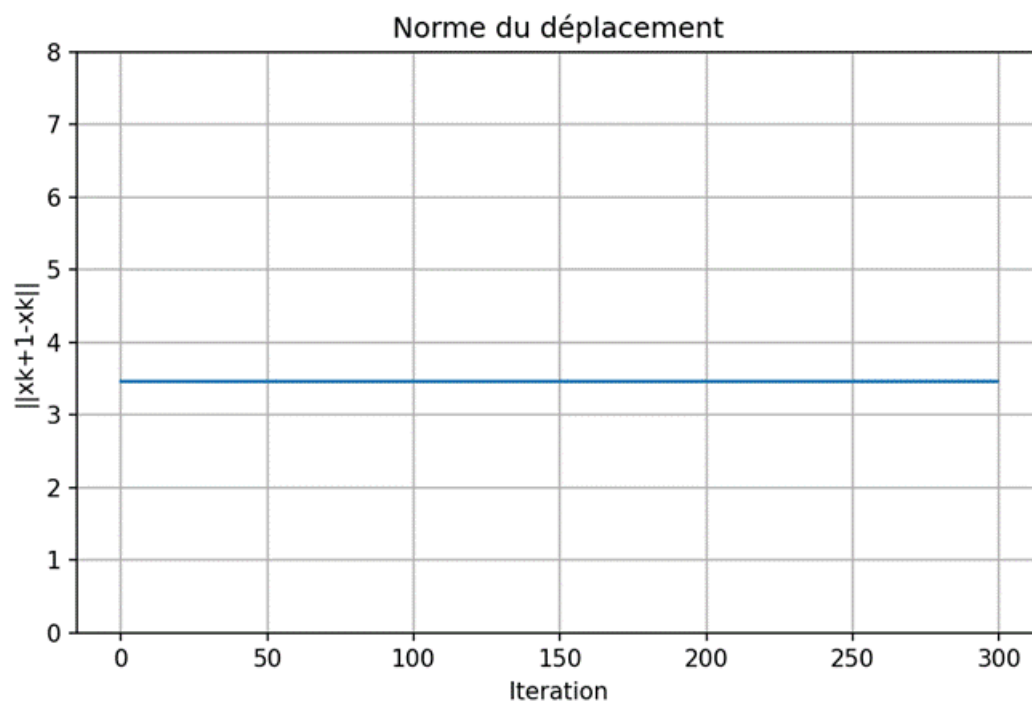
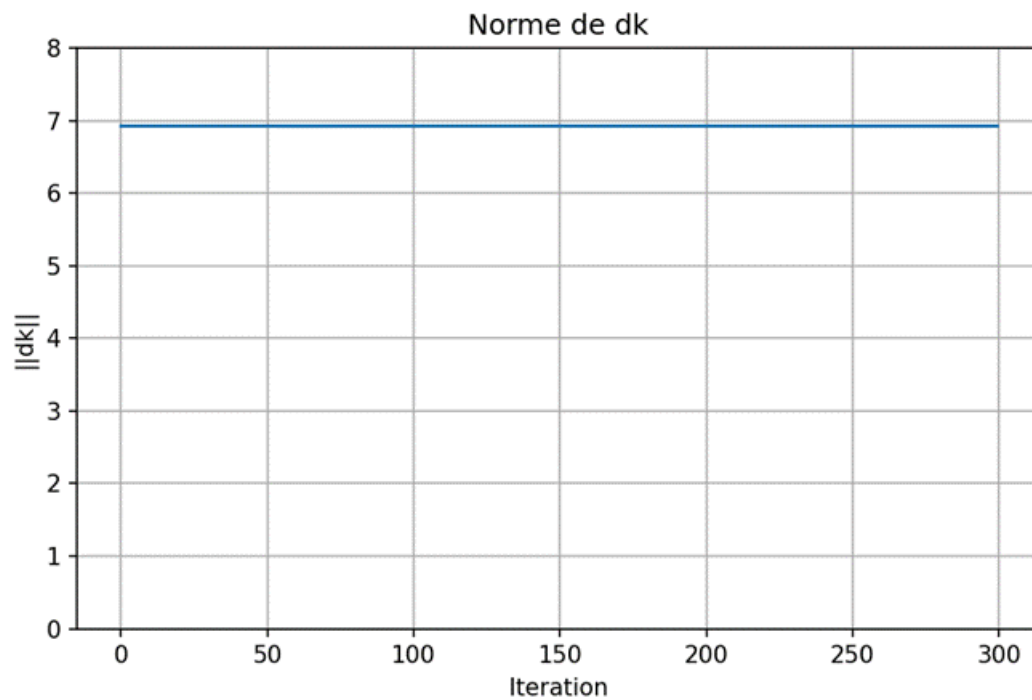
Observation

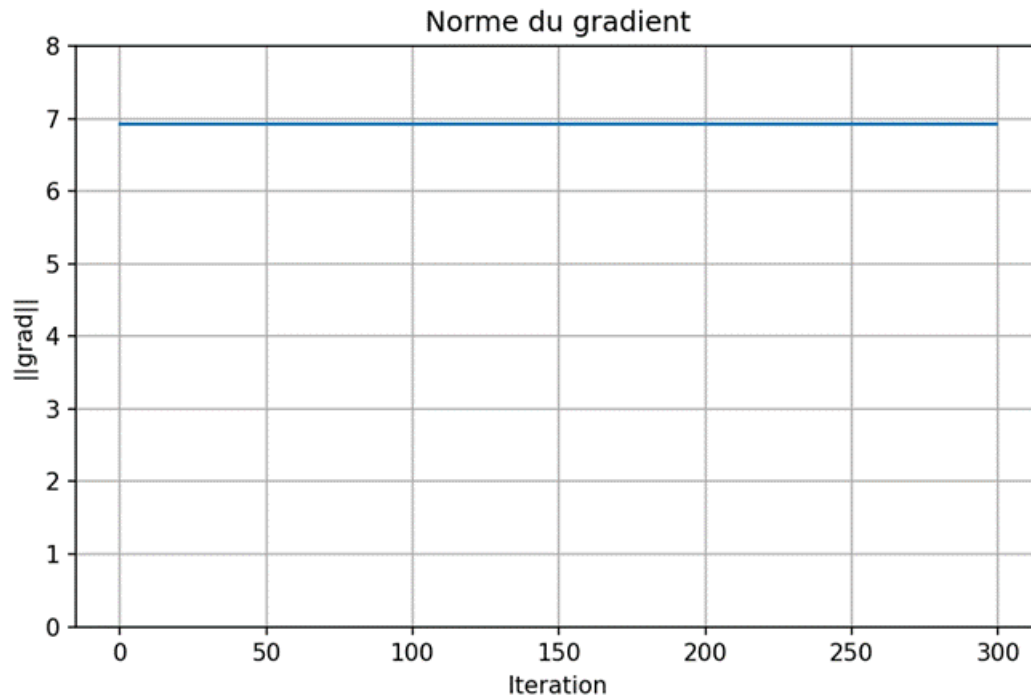
Cette fois, contrairement au Travail 1 et au Travail 2 :

- le point oscille indéfiniment entre (1, 1, 1) et (-1, -1, -1) ;
- la norme du gradient reste constante (≈ 6.9282) et ne tend pas vers 0 ;
- la méthode ne converge pas (sans pour autant diverger).

Explication : la matrice hessienne de f est constante et sa plus grande valeur propre vaut $\lambda_{\max} = 4$. Pour un pas fixe, la méthode du gradient converge si $0 < \alpha < 2/\lambda_{\max} = 0.5$. Ici $\alpha = 0.5$ est exactement le pas critique : la composante du point selon la direction propre (1, 1, 1) est multipliée par $(1 - \alpha\lambda_{\max}) = -1$ à chaque itération, d'où l'oscillation.

Représentation graphique :





Cas $\alpha = 0.5$ et $\text{tol} = 10^{-3}$

Nombre d'itérations : précision jamais atteinte

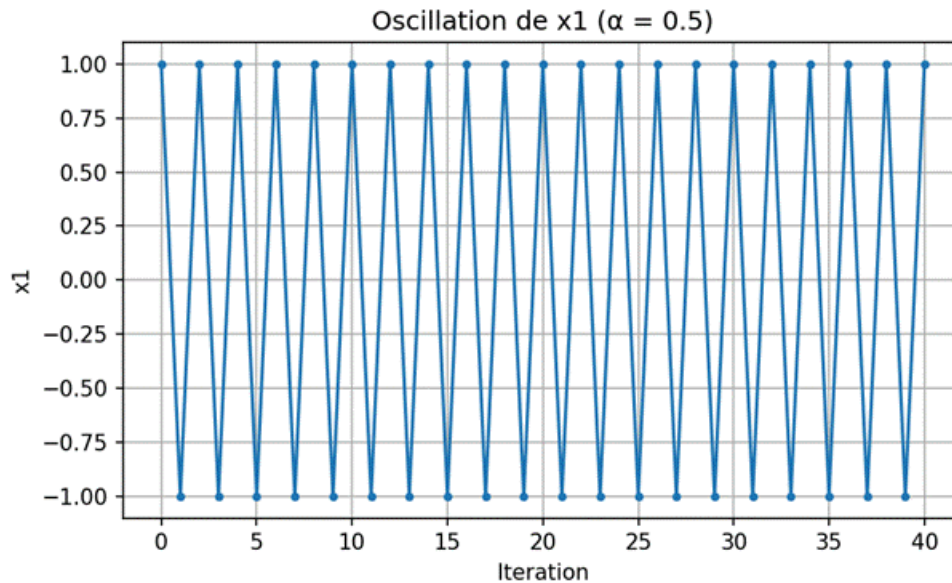
($\| \text{grad } f(x_k) \| = 6.9282$ pour tout k – arrêt forcé après 10 000 itérations)

Cas $\alpha = 0.5$ et $\text{tol} = 10^{-5}$

Nombre d'itérations : précision jamais atteinte

($\| \text{grad } f(x_k) \| = 6.9282$ pour tout k – arrêt forcé après 10 000 itérations)

Le critère d'arrêt n'est jamais vérifié : la norme du gradient ne diminue pas, comme le montre l'oscillation ci-dessous.



Complément : convergence avec un pas plus petit ($\alpha = 0.4$)

En choisissant un pas vérifiant la condition de convergence $\alpha < 2/\lambda_{\max} = 0.5$, par exemple $\alpha = 0.4$, la méthode converge bien vers le minimum $(0, 0, 0)$:

Cas $\alpha = 0.4$ et $\text{tol} = 10^{-3}$

Nombre d'itérations : 18

Temps d'exécution : 0.00009655952453613281 secondes

Cas $\alpha = 0.4$ et $\text{tol} = 10^{-5}$

Nombre d'itérations : 27

Temps d'exécution : 0.0001544952392578125 secondes

Trajectoire des itérés ($\alpha = 0.4$, $\text{tol} = 10^{-5}$) :

